

State-of-the-Art Agentic Repository Discovery Harnesses for Senior-Engineer-Like Codebase Understanding

Executive summary

Your use-case is not “autonomous coding” in the broad sense. It is a narrower, higher-leverage problem: **automated repository discovery** that builds a dependable understanding of an unknown codebase, then emits structured artefacts that later topic-specific memory compilers can refine into durable agent and human memory. In practice, that means the harness must do three things well: retrieve the right evidence, organise that evidence into navigable structure, and expose uncertainty rather than hallucinating architecture. Sourcegraph’s context-engine work, Aider’s repo-map design, Anthropic’s guidance on agent-computer interfaces, and the latest context-retrieval benchmark literature all point in the same direction: the best results come less from “more autonomy” and more from **better interfaces, better retrieval, and better output contracts**. ¹

The strongest current pattern for repository discovery is therefore a **hybrid harness**: a deterministic bootstrap pass to map the repository, followed by bounded graph-driven exploration rather than a fully free-form agent loop. This matches Aider’s approach of building a compact global repo map and ranking relevance under a token budget, Sourcegraph’s framing of the context engine as a specialised retrieval component, and Anthropic’s recommendation to keep systems simple until more complexity measurably improves outcomes. It also fits recent evidence from ContextBench, which finds that more elaborate coding-agent scaffolding often yields only marginal context-retrieval gains and that current agents still over-retrieve noisy context. ²

For implementation, the most reusable “bricks” are not monolithic agent frameworks. They are the lower layers: **tree-sitter** for cross-language parsing and AST queries, **ripgrep** for exact search, **universal ctags** and optionally **LSP/LSIF** for symbols and references, **ast-grep** for structural queries, **CodeQL/Semgrep/Joern** for higher-signal static facts, and **NetworkX or Neo4j** for representing repository areas as a graph. If you need a centralised code search back-end, **Zoekt/Sourcegraph** or **OpenGrok** are more directly useful than pure vector databases. Dense retrieval layers such as **Qdrant** or **LanceDB** are helpful, but should be secondary to exact, structural, and dependency-aware retrieval for first-pass codebase discovery. ³

On orchestration, the best current fit is **single orchestrator first, subagents second**. OpenAI’s Agents SDK, Vercel’s Eve, Vercel AI SDK, LangGraph, and AutoGen all support multi-step tool loops and different flavours of delegation, but Anthropic’s production guidance and research suggest you should only add multi-agent breadth where the task genuinely parallelises. Anthropic explicitly notes that many coding tasks do not parallelise as cleanly as research, even though multi-agent systems can outperform on breadth-first exploration tasks. For repo discovery, that means one principal “staff engineer” agent should own the run, while short-lived subagents can be used only for independent repository areas or evidence-collection branches. ⁴

The recommended output is **not** a single long narrative summary. It should be a structured package: an area graph of file-groups, symbol and dependency evidence, reading plans, conventions, hotspots, unanswered questions, and confidence scores. This is conceptually close to Anthropic's **Agent Skills** and the open **Agent Skills** format: progressive disclosure, portable folders, and focused task-specific knowledge loaded on demand. The discovery harness should emit memory seeds in this style so that later compilers can transform them into topic memories without having to rediscover the repository from scratch. ⁵

My bottom-line recommendation is to prototype a harness with this stack: **OpenAI Agents SDK or LangGraph for orchestration; tree-sitter + ripgrep + ctags + ast-grep as the discovery core; optional LSP/LSIF when available; NetworkX as the internal graph layer; Qdrant or LanceDB only for secondary retrieval; and GraphRAG-like hierarchical summarisation only after the code graph exists.** This will get you closest to a Senior-Engineer-like discovery loop while keeping complexity, cost, and evaluation manageable. ⁶

Use-case definition and design criteria

The use-case can be stated precisely as follows: **given an arbitrary repository or repository set, automatically build enough trustworthy context to explain the codebase's structure, conventions, critical flows, and topic boundaries, and emit that knowledge as machine-readable and human-readable memory seeds for later coding agents and topic compilers.** The important assumptions are that languages, frameworks, repository size, monorepo status, and CI/CD shape are all unspecified at intake, so the harness must degrade gracefully from "tiny single-service repo" to "large polyglot mono- or multi-repo estate". That requirement pushes the design toward language-agnostic indexing and progressive disclosure, not toward stack-specific hard-coding. ⁷

A Senior Engineer discovering a codebase does not start by reading every file. They establish the terrain: repository layout, build/runtime boundaries, public entry points, dependency edges, conventions, test surfaces, and "where the truth lives". Aider's repo map encodes exactly that idea by giving the model a compact, ranked overview of symbols and signatures before deeper inspection. Sourcegraph's context-engine framing reaches the same conclusion from another angle: retrieval is a specialised subsystem whose job is to provide relevant contextual snippets, not indiscriminately dump code into the prompt. ⁸

That implies four design criteria for your harness. First, **evidence-first retrieval**: exact search, symbol lookup, AST queries, and dependency graphs should outrank semantic retrieval for first-pass discovery. Second, **progressive disclosure**: the agent should maintain a compact world model and expand detail only when a branch becomes relevant, which is exactly the logic behind Agent Skills' staged loading and Anthropic's advice to keep tool surfaces selective. Third, **bounded exploration**: the harness must carry explicit budgets for depth, breadth, tokens, and wall-clock steps. Fourth, **structured emissions**: the run must end in artefacts that can be consumed by later agents, not merely a terminal transcript. ⁹

One important consequence of recent research is that "more agentic" is not automatically "more state of the art" for this use-case. ContextBench finds that sophisticated scaffolding often improves final task framing less than expected at the retrieval layer, and Anthropic's production guidance explicitly recommends starting simple, making planning transparent, and designing the agent-computer interface carefully. For repo discovery, the state of the art is therefore not a maximalist swarm. It is a strong retrieval and evidence harness that optionally uses delegation where the repository naturally decomposes. ¹⁰

State-of-the-art harnesses and framework patterns

The current landscape splits into two groups: **general agent harnesses** and **repo-aware coding-agent harnesses**. Among general harnesses, **OpenAI's Responses API and Agents SDK** are now one of the clearest reference designs for typed tool loops, resumable state, handoffs, tracing, and container-based sandbox work. OpenAI explicitly positions the Responses API as the agentic primitive for multi-tool loops and the Agents SDK as the code-first layer when the application owns orchestration, approvals, and state. That is highly relevant to repository discovery because discovery is fundamentally a multi-step tool loop with resumable intermediate products. ¹¹

LangGraph and LangChain remain strong when you want orchestration control rather than platform coupling. LangChain's own documentation defines a harness as the machinery that gets the model the right context at the right time, and LangGraph focuses on durable execution, persistence, memory, streaming, and human-in-the-loop control. That is a very good conceptual fit for discovery runs that may need pauses, approvals, or resumptions, although it leaves more implementation responsibility to you than platform-native SDKs do. ¹²

Vercel AI SDK and the new **Eve** framework are especially interesting because they are filesystem-first and progressive-disclosure oriented. Vercel AI SDK's `ToolLoopAgent`, `stopWhen`, and `prepareStep` encode a clean, explicit multi-step reasoning-and-acting loop. Eve pushes the idea further by making an agent effectively a directory: Markdown instructions and skills, TypeScript tools, subagents, schedules, connections, and isolated sandboxes. For a repository discovery harness, that "agent as inspectable folder" pattern is attractive because it keeps behaviour legible and makes playbooks reusable; the downside is maturity, because Eve is explicitly in beta and very new. ¹³

Anthropic's documentation and engineering patterns are among the best primary sources for harness design principles. Anthropic defines the "augmented LLM" as the base unit, argues that frameworks help but can hide the real behaviour, recommends simplicity and transparency, and emphasises careful agent-computer interface design and tool documentation. Anthropic's tool-use documentation is also one of the clearest explanations of the agentic loop and of where tool execution happens. For repository discovery, Anthropic's newer **Agent Skills** work is particularly relevant: it formalises progressive disclosure of procedural knowledge through folders, metadata, and on-demand loading, which maps closely to the memory artefacts your later compilers will want. ¹⁴

Microsoft AutoGen still matters historically and architecturally, especially for event-driven multi-agent patterns and extensions such as MCP workbenches and Docker code executors. However, Microsoft now describes the open-source repo as in maintenance mode, which lowers its recommendation for a new build unless you already have team familiarity with it. Its abstractions are still useful for thinking about conversational and event-driven agent collaboration, but I would not choose it first for a fresh repo-discovery harness in 2026. ¹⁵

On the repo-aware side, **SWE-agent** and **OpenHands** are more directly relevant than many general frameworks. The key contribution of SWE-agent is not a clever planner but a specialised **agent-computer interface** that materially improves repository navigation, editing, and tool use; its documentation also exposes a relatively simple YAML-controlled configuration surface. OpenHands goes further toward a composable software-agent runtime with local or ephemeral workspaces, Docker/Kubernetes execution, and a dedicated agent server. These are exemplary references for what a discovery harness needs from its execution environment, even if your use-case stops before code modification. ¹⁶

For codebase understanding specifically, **Sourcegraph Cody** and **Sourcegraph's MCP/context-engine work** are among the most relevant industry references. Sourcegraph's own material frames context retrieval as the critical search component of an AI coding assistant, explains that Cody uses RAG over codebase context, and describes scaling from local file to local repo to remote multi-repo context, including very large repository estates indexed through Sourcegraph and Zoekt. This is directly aligned with your discovery problem, because it treats understanding as retrieval and ranking over a code graph rather than as raw token stuffing. ¹⁷

Aider deserves a special mention despite being a product rather than a framework paper. Its repo-map design is the closest production pattern to a Senior-Engineer-like reconnaissance pass: build a compact whole-repo symbol map, then let the model request deeper inspection where needed, while ranking what fits into the active token budget. That is a very strong design cue for your discovery harness. ¹⁸

The academic patterns remain foundational but should be treated as patterns, not drop-in harnesses. **ReAct** established the interleaving of reasoning and actions; **Toolformer** showed that tool selection can be learned/self-supervised; **Reflexion** argued for explicit self-reflection and episodic memory; **BabyAGI** popularised task loops and now experiments with graph-based function storage. These ideas remain useful, but the practical state of the art for repo discovery comes from combining them with stronger tool interfaces, retrieval, and observability rather than adopting any one paper literally. ¹⁹

A final, important caution comes from the newest evidence. Anthropic's own multi-agent research system shows strong gains for breadth-first search tasks through parallel subagents, but also notes that many coding tasks do not parallelise as cleanly. ContextBench independently finds that extra scaffolding often does not buy much in retrieval quality. For repo discovery, the implication is clear: use multi-agent breadth as an optimisation for independent code areas, not as the default architecture. ²⁰

Reusable implementation bricks and recommended component stack

The most valuable reusable bricks are the ones that help the harness behave like a careful engineer who first maps, then drills down. **tree-sitter** is the best default parser layer because it is incremental, language-agnostic across many grammars, and queryable via S-expression patterns; **ripgrep** is the best low-friction exact-search primitive; **universal ctags** remains a very practical symbol extractor across languages and underpins Aider's repo-map approach; and **LSP/LSIF** should be mounted when available because definitions, references, and symbol navigation are native discovery primitives. In other words, the baseline stack should be structural and symbolic before it is semantic. ²¹

For structural search and conventions mining, **ast-grep** is a strong addition because it gives you AST-shaped queries and rewrites rather than regex over text. For architecture or dependency maps, **dependency-cruiser** and **madge** are good JS/TS-specific add-ons, while **NetworkX** or **Neo4j** provide the graph substrate for a language-agnostic "areas graph". For centralised code search at scale, **Zoekt/Sourcegraph** or **OpenGrok** are the more relevant search/indexing engines. OpenGrok is mature and useful for cross-reference search; Zoekt is a fast trigram-based engine that already powers Sourcegraph's search stack. ²²

For static facts that go beyond simple search, **Semgrep**, **CodeQL**, and **Joern** are especially useful. Semgrep gives light-weight, multi-language pattern scanning; CodeQL gives deep queryable semantic analysis backed by a mature ecosystem; Joern gives code property graphs for cross-language program analysis. These are not just "security tools" in this context. They are high-signal fact extractors for conventions, dangerous hotspots, public surface identification, and data/control-flow hints. ²³

For semantic retrieval and long-lived memory stores, **Qdrant**, **LanceDB**, and **Weaviate** all work. Qdrant explicitly supports dense and sparse vectors with filtering and client/server APIs; LanceDB is attractive if you want an embedded retrieval layer and one-table storage of raw data plus embeddings plus metadata; Weaviate is a stronger “AI database” when you want a broader managed feature set. My recommendation for discovery is to use one of these only as a **secondary retrieval plane** for summaries, docs, README fragments, and previously compiled memory artefacts—not as the primary way of finding relevant code. Both Sourcegraph’s context work and ContextBench support that prioritisation. ²⁴

For memory compilation and navigable summaries, **GraphRAG** is promising, but it should come after you already have a code graph and evidence graph. GraphRAG is designed to extract structured knowledge graphs, build community hierarchies, and summarise them for retrieval. That is very relevant for human-readable and agent-readable memory layers, but less suitable as the first discovery step over raw code. Use it to summarise discovered areas and conventions, not to replace parsing and symbol extraction. ²⁵

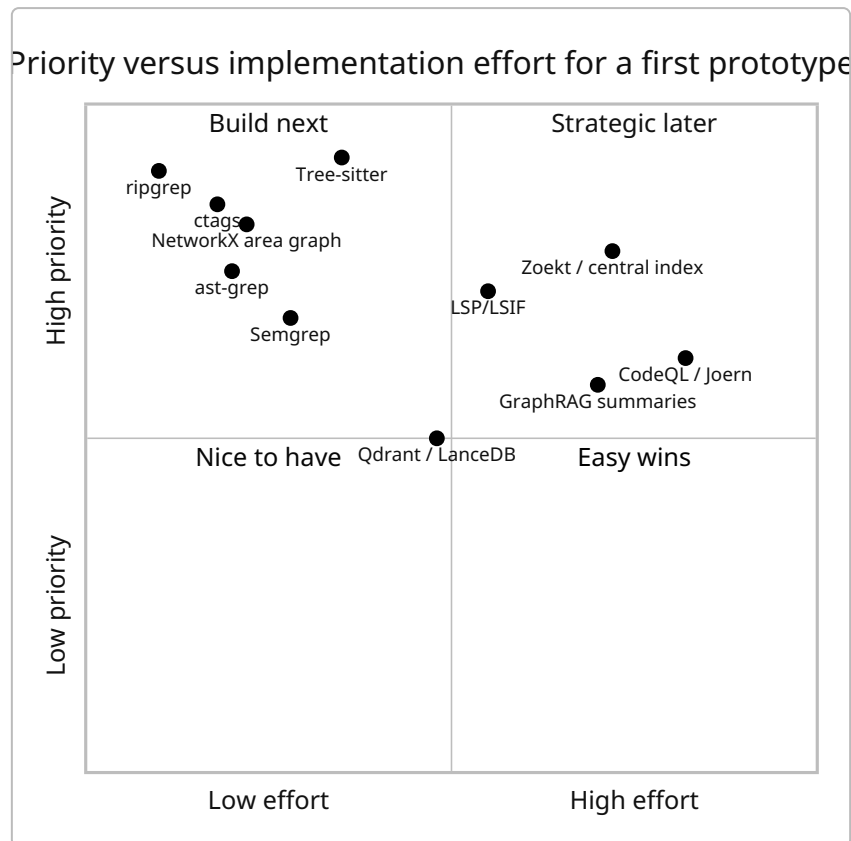
The table below compares the most reusable candidates for a first harness.

Name	Purpose	License	Maturity	Language	Integration complexity	Notes
Tree-sitter	Incremental parsing and AST queries	MIT	High	Multi-language grammars	Medium	Best default parser layer for symbol blocks, import edges, and chunking. Official docs expose incremental parsing and query syntax. ²⁶
ripgrep	Fast exact and regex code search	MIT / Unlicense	High	Rust CLI, language-agnostic target	Low	Ideal first-pass search primitive; respects <code>.gitignore</code> by default and is extremely easy to embed in a harness. ²⁷
Universal Ctags	Cross-language symbol extraction	GPL-2.0-compatible project lineage; use official project terms	High	C, many target languages	Low	Very strong low-cost symbol indexer; Aider uses it to build its repo map. ²⁸
LSP / LSIF	Definitions, references, symbol navigation	Open protocol / mixed server licences	High where servers exist	Multi-language	Medium	Use opportunistically: strongest when a repo already has stable language-server support. ²⁹

Name	Purpose	License	Maturity	Language	Integration complexity	Notes
ast-grep	Structural code search	MIT	High	Rust CLI, many targets	Low	Best “grep but AST-aware” brick for conventions and pattern discovery. ³⁰
Sourcegraph Zoekt / Cody context stack	Code search and context retrieval at scale	Apache-2.0 for Zoekt; Cody extension Apache-2.0; platform mixed/commercial	High	Go / TypeScript / platform	High	Reference design for large-scale context engines and multi-repo retrieval; very relevant if you need central indexing. ³¹
OpenGrok	Search and cross-reference engine	CDDL	High	Java	Medium	Mature cross-reference/search engine; useful when you want self-hosted search rather than embeddings. ³²
Semgrep	Pattern-based static analysis	OSS community edition available	High	Multi-language	Low	Good for conventions, anti-patterns, risky hotspots, and codified org rules. ³³
CodeQL	Semantic code analysis and query engine	Open-source repo / free for OSS and research	High	Multi-language	High	Best when you need deep semantic facts and rich queryability; expensive to operationalise broadly. ³⁴
Joern	Code property graph analysis	Apache-2.0	Medium-High	Scala platform, multi-language targets	High	Powerful if you want dependency/dataflow graphs beyond ordinary symbol indices. ³⁵
Qdrant	Vector and hybrid retrieval store	Apache-2.0	High	Rust server, multi-client	Medium	Good secondary retrieval plane for summaries and memory artefacts, not first-pass code discovery. ³⁶
LanceDB	Embedded vector/hybrid retrieval	Apache-2.0	Medium-High	Embedded, multi-SDK	Low-Medium	Attractive for local-first prototypes and mixed metadata+embedding tables. ³⁷

Name	Purpose	License	Maturity	Language	Integration complexity	Notes
NetworkX	In-memory graph construction and algorithms	BSD-3-Clause	High	Python	Low	Best first graph substrate for area graphs, ranking, centrality, and planning. ³⁸
Neo4j	Persistent property graph storage	Commercial/ open offerings	High	JVM / DB	Medium-High	Better once the discovery artefacts need persistent graph queries across repos or teams. ³⁹
GraphRAG	Hierarchical graph-based summarisation	MIT-style open-source project	Medium	Python	Medium-High	Good for summarising discovered areas into navigable memory, but not for initial raw-code mapping. ⁴⁰
Aider repo map	Reference pattern for compact global code context	Apache-2.0	High	Python	Low concept / N/A as a pattern	Not a brick to embed verbatim so much as a design pattern to copy: symbol map + graph ranking + selective expansion. ⁴¹

A practical priority order for reuse is therefore: **tree-sitter, ripgrep, ctags, NetworkX, ast-grep, Semgrep, optional LSP/LSIF, then a central index/search back-end if scale demands it**. Only after those are in place would I add vector retrieval and GraphRAG-style summarisation. That ordering follows the strongest current evidence on repository context quality, interpretability, and operational simplicity. ⁴²



Proposed harness contract

The harness contract should be designed around the discovery job, not around generic chat. The **required context inputs** should be: repository locator or mounted path; optional commit or branch; optional seed files or seed questions; budget constraints; any known org conventions or architectural doctrines; and optional execution affordances such as “tests may run” or “network disabled”. If the repository set is large, the run spec should also allow explicit scope boundaries, for example “service X only” or “areas touching auth and billing”. This mirrors the way mature harnesses separate the agent definition from runtime configuration, tools, and resumable state. ⁴³

The **tool surface** should be small, sharp, and semantically named. Anthropic’s tool-design guidance is highly relevant here: tools should have clear boundaries, return high-signal context, and reduce rather than increase prompt burden. For this use-case, the minimum tool set is: repository listing, file read, exact search, structural search, symbol index query, reference/definition query, dependency/area graph builder, static-rules scan, git-history query, test/CI metadata read, and memory-artifact write. I would avoid giving the discovery agent fully general shell execution by default; shell or sandbox tools should be gated behind explicit policy because discovery mostly benefits from read-heavy interfaces. ⁴⁴

The **expected outputs** should be structured artefacts rather than prose-first summaries. At minimum, each run should emit: a repository overview; an area graph of file-groups; entry points and integration points; a symbol index for the most central entities; conventions and norms; hotspots and risky areas; a reading plan; unresolved questions; and confidence metrics. OpenAI’s Agents SDK result surfaces are a useful analogy here: discovery should return both the “final answer” and the resumable or replayable intermediate state. In your harness, that means the output should preserve enough evidence and references that later memory compilers can continue from it without re-running discovery. ⁴⁵

A good default data contract is shown below.

```
{
  "run_id": "disc_2026_06_18_001",
  "repo": {
    "root": "/workspace/repo",
    "revision": "HEAD",
    "scope": ["."],
    "language_hints": [],
    "monorepo": null
  },
  "budgets": {
    "max_steps": 80,
    "max_files_read": 300,
    "max_tokens_for_context": 120000,
    "allow_shell": false
  },
  "seeds": {
    "questions": [],
    "files": [],
    "topics": []
  }
}
```

```
{
  "area_id": "area.auth.http",
  "label": "Authentication HTTP surface",
  "kind": "code_area",
  "files": [
    "src/auth/routes.ts",
    "src/auth/middleware.ts",
    "src/auth/service.ts"
  ],
  "entrypoints": [
    "registerAuthRoutes",
    "authMiddleware"
  ],
  "owned_symbols": [
    "AuthService",
    "TokenVerifier"
  ],
  "depends_on": [
    "area.shared.config",
    "area.user.domain"
  ],
  "evidence": [
    {
      "type": "symbol",
      "path": "src/auth/routes.ts",
    }
  ]
}
```

```

    "symbol": "registerAuthRoutes",
    "lines": "12-74"
  }
],
"conventions": [
  "HTTP middleware composes service objects rather than calling db code directly"
],
"uncertainties": [
  "Token refresh flow inferred but not fully confirmed from tests"
],
"recommended_next_reads": [
  "src/auth/tests/routes.test.ts",
  "src/user/domain/user.ts"
],
"confidence": 0.78
}

```

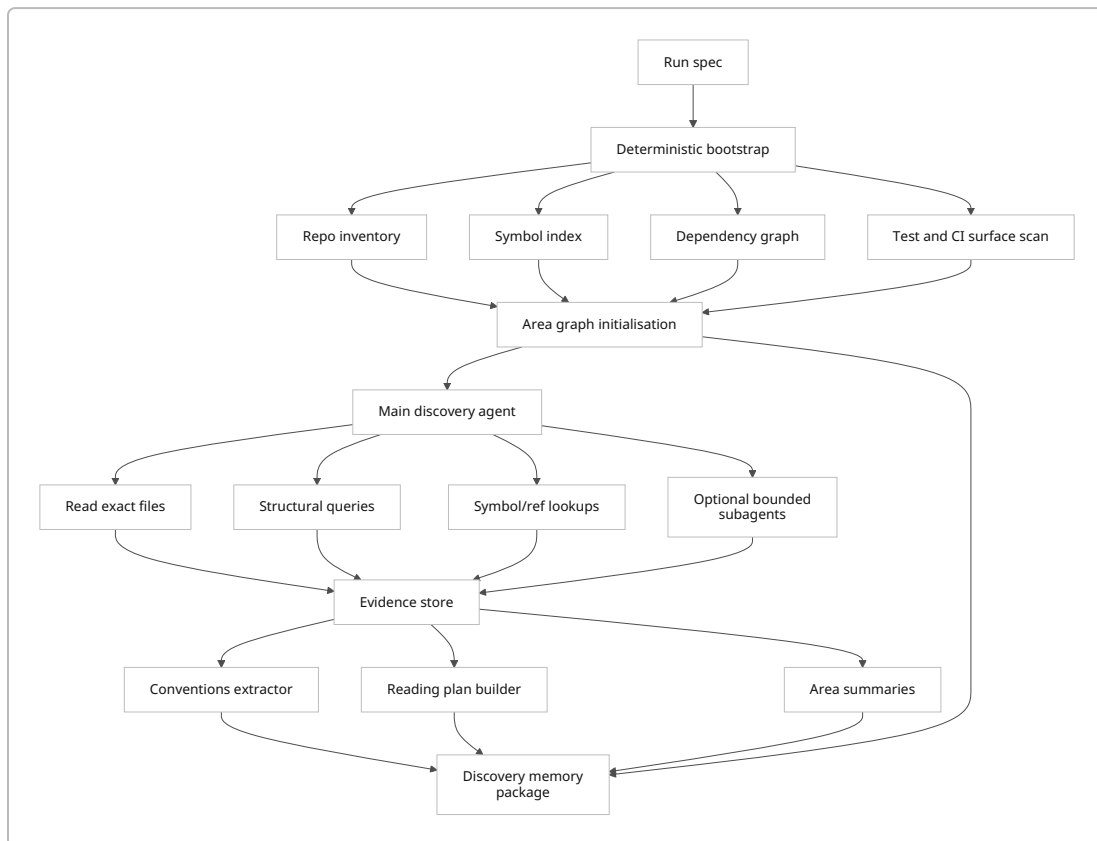
```

{
  "run_id": "disc_2026_06_18_001",
  "summary": {
    "system_shape": "multi-package service platform",
    "primary_entrypoints": ["cmd/api/main.go", "web/app.tsx"],
    "build_systems": ["pnpm", "Go modules"],
    "confidence": 0.74
  },
  "areas": [],
  "edges": [],
  "conventions": [],
  "hotspots": [],
  "reading_plan": [],
  "open_questions": [],
  "metrics": {
    "files_seen": 0,
    "symbols_indexed": 0,
    "areas_built": 0,
    "test_surfaces_found": 0
  }
}

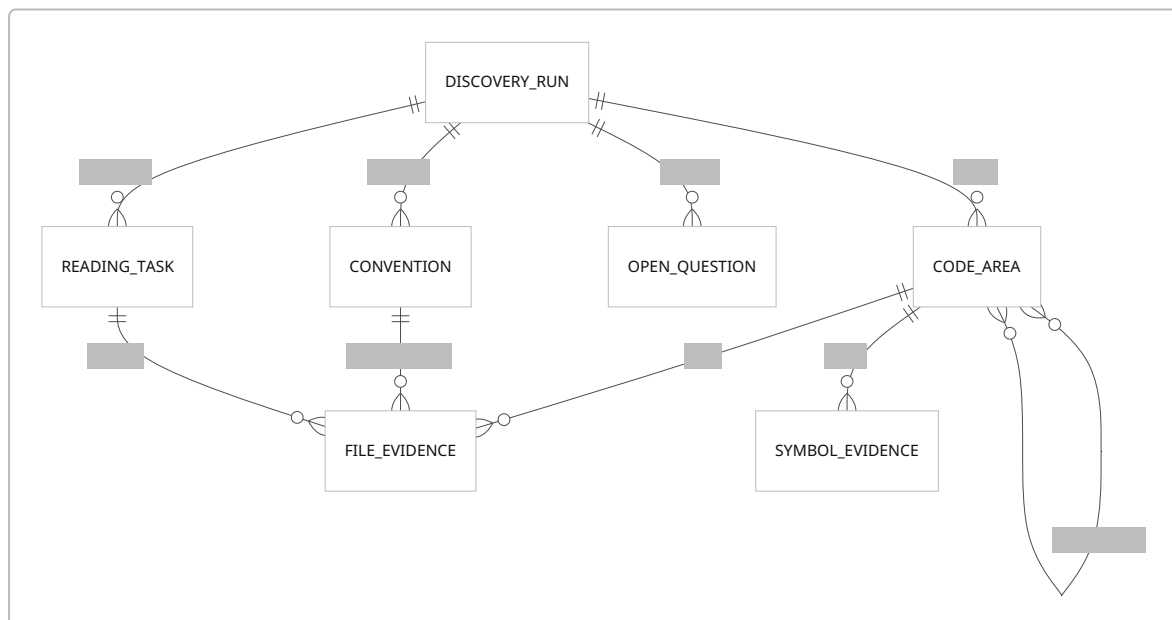
```

The best **orchestration pattern** for this use-case is a hybrid. Start with a deterministic bootstrap: detect languages, parse manifests, infer workspace boundaries, enumerate entry points, build a global symbol/dependency sketch, and seed centrality scores. Then switch to graph-driven exploration in which the main agent chooses the next area to inspect based on uncertainty, centrality, and task coverage. Use subagents only for independent branches such as “frontend shell”, “payments service”, or “infra directory”, and merge their outputs back into the main graph. That combines Sourcegraph’s specialised retrieval mindset, Aider’s graph-ranked repo map, OpenAI’s manager-versus-specialist orchestration choices, and Anthropic’s caution about unnecessary complexity. ⁴⁶

The following architecture is what I would recommend.



And this is the memory-artifact relationship that later topic compilers can build on.



Implementation roadmap and quick prototype plan

The fastest sound prototype is a **three-layer build**. In the first layer, implement the deterministic bootstrap: repository inventory, manifest parsing, language detection, exact search, symbol extraction, and dependency sketching. Use **ripgrep**, **tree-sitter**, **universal ctags**, and **NetworkX** here. In the second layer, add the orchestration harness and bounded exploration loop. In the third layer, add

structured output artefacts, evaluations, and only then optional semantic retrieval or GraphRAG-derived summaries. This sequencing follows the broad consensus in the sources that better interfaces and retrieval matter more than premature orchestration complexity. ⁴⁷

For orchestration, choose **OpenAI Agents SDK** if you want a modern Python/TypeScript code-first runtime with built-in tracing, resumable state, handoffs, and growing sandbox support; choose **LangGraph** if you want maximum control and framework independence; choose **Vercel AI SDK/Eve** if your implementation is TypeScript-first and you value inspectable filesystem-first agents. For a research-heavy prototype, **OpenHands SDK** is also a strong reference if you want reusable code-agent execution primitives, but it is more than you need for a read-mostly discovery harness. ⁴⁸

A realistic quick prototype plan is:

First, build the **indexing micro-pipeline**. Parse the repository tree, collect manifests, create a global symbol table, compute import/dependency edges, and emit an initial area graph based on directories plus dependency clustering. Second, add an orchestrator that can ask: “What is the most uncertain but central area?” and follow a reading plan of 5–10 files per area, retrieving exact snippets and symbol blocks rather than entire files where possible. Third, add convention extraction passes over tests, lint configs, CI files, and repeated structural patterns. Fourth, emit the memory package as JSON plus a readable Markdown summary. Fifth, evaluate on a handful of internal and open repositories using task-based rubrics such as “can a new engineer find entrypoints, build commands, extension points, and auth flow in under N artefacts?”. Anthropic’s and LangSmith’s evaluation guidance, plus ContextBench’s process orientation, support making evaluation a first-class concern from the start. ⁴⁹

The main risks are not model quality. They are **over-retrieval**, **unstable area partitioning**, **monorepo boundary confusion**, **tool sprawl**, and **false certainty in summaries**. ContextBench shows that current systems tend to favour recall over precision; Anthropic warns that overlapping or weakly named tools make agents worse; and large-codebase context systems such as Sourcegraph succeed by specialising the retrieval layer and exposing scale-aware search rather than relying on generic semantic search alone. Your prototype should therefore instrument file reads, retrieval provenance, area-confidence scores, and “used versus merely seen” evidence metrics from day one. ⁵⁰

My prioritised reusable stack for the first implementation is:

First priority: tree-sitter, ripgrep, universal ctags, NetworkX, and a code-first harness such as OpenAI Agents SDK or LangGraph. These are the foundations of the discovery loop. ⁵¹

Second priority: ast-grep, Semgrep, optional LSP/LSIF, and a small evidence store. These improve structural precision and conventions mining. ⁵²

Third priority: Zoekt/OpenGrok or Sourcegraph integration if repository scale requires central indices; Qdrant or LanceDB only for secondary retrieval over summaries and compiled memories. ⁵³

Fourth priority: GraphRAG-style hierarchical summaries and richer static-analysis layers such as CodeQL or Joern, once the baseline harness is already stable and evaluable. ⁵⁴

Open questions and limitations

A few points remain inherently context-dependent. The right choice between **OpenAI Agents SDK**, **LangGraph**, and **Vercel AI SDK/Eve** depends partly on your preferred implementation language,

hosting model, and how much runtime abstraction you want versus how much you want to own yourself. The sources are strong on capabilities, but they do not answer your team's internal preference trade-offs. ⁵⁵

There is also no single universal answer for **area graph construction** in polyglot monorepos. JS/TS has mature module-graph tools such as dependency-cruiser and madge, while other languages may rely more on manifests, imports, build files, or language servers. That is why the report recommends a language-agnostic baseline from tree-sitter, ctags, exact search, and graph assembly rather than one monolithic dependency extractor. ⁵⁶

Finally, evaluation for repository discovery is still an emerging area. ContextBench is the strongest recent signal because it evaluates context retrieval process rather than only end-to-end issue resolution, but it is still a benchmark rather than a turnkey internal QA system. You will still need your own rubric for "Senior-Engineer-like understanding" and your own internal truth sets for architecture, conventions, and reading plans. ⁵⁷

¹ ¹⁷ ⁴⁶ Lessons from building AI coding assistants: Context retrieval and evaluation | Sourcegraph
<https://sourcegraph.com/blog/lessons-from-building-ai-coding-assistants-context-retrieval-and-evaluation>

² ⁸ ¹⁸ ⁴¹ ⁴² Repository map | aider
<https://aider.chat/docs/repomap.html>

³ ²¹ ²⁶ ⁵¹ Tree-sitter
https://github.com/tree-sitter/tree-sitter?utm_source=chatgpt.com

⁴ ⁴³ ⁴⁸ ⁵⁵ Agents SDK | OpenAI API
<https://developers.openai.com/api/docs/guides/agents>

⁵ Equipping agents for the real world with Agent Skills \ Anthropic
<https://www.anthropic.com/engineering/equipping-agents-for-the-real-world-with-agent-skills>

⁶ The next evolution of the Agents SDK | OpenAI
<https://openai.com/index/the-next-evolution-of-the-agents-sdk/>

⁷ How Cody provides remote repository awareness for codebases of every size | Sourcegraph
<https://sourcegraph.com/blog/how-cody-provides-remote-repository-context>

⁹ Agent Skills Overview - Agent Skills
<https://agentskills.io/home>

¹⁰ ⁵⁰ ⁵⁷ ContextBench: A Benchmark for Context Retrieval in Coding Agents
<https://arxiv.org/html/2602.05892v2>

¹¹ Migrate to the Responses API | OpenAI API
<https://developers.openai.com/api/docs/guides/migrate-to-responses>

¹² Agents - Docs by LangChain
<https://docs.langchain.com/oss/python/langchain/agents>

¹³ AI SDK Core: ToolLoopAgent
<https://ai-sdk.dev/docs/reference/ai-sdk-core/tool-loop-agent>

¹⁴ Building Effective AI Agents \ Anthropic
<https://www.anthropic.com/research/building-effective-agents>

¹⁵ AutoGen — AutoGen
<https://microsoft.github.io/autogen/stable//index.html>

- 16 SWE-agent: Agent-Computer Interfaces Enable Automated ...
https://arxiv.org/abs/2405.15793?utm_source=chatgpt.com
- 19 Synergizing Reasoning and Acting in Language Models
https://arxiv.org/abs/2210.03629?utm_source=chatgpt.com
- 20 How we built our multi-agent research system \ Anthropic
<https://www.anthropic.com/engineering/multi-agent-research-system>
- 22 30 52 ast-grep/ast-grep: ⚡ A CLI tool for code structural search, ...
https://github.com/ast-grep/ast-grep?utm_source=chatgpt.com
- 23 33 Semgrep Docs - Semgrep
https://docs.semgrep.dev/?utm_source=chatgpt.com
- 24 36 Qdrant Overview
https://qdrant.tech/documentation/overview/?utm_source=chatgpt.com
- 25 Welcome - GraphRAG
https://microsoft.github.io/graphrag/?utm_source=chatgpt.com
- 27 47 ripgrep recursively searches directories for a regex pattern ...
https://github.com/burntsushi/ripgrep?utm_source=chatgpt.com
- 28 Home · Universal Ctags
https://ctags.io/?utm_source=chatgpt.com
- 29 Official page for Language Server Protocol
https://microsoft.github.io/language-server-protocol/?utm_source=chatgpt.com
- 31 53 sourcegraph/zoekt: Fast trigram based code search
https://github.com/sourcegraph/zoekt?utm_source=chatgpt.com
- 32 OpenGrok is a fast and usable source code ...
https://github.com/oracle/opengrok?utm_source=chatgpt.com
- 34 About CodeQL - GitHub
https://codeql.github.com/docs/codeql-overview/about-codeql/?utm_source=chatgpt.com
- 35 Joern - The Bug Hunter's Workbench
https://github.com/joernio/joern?utm_source=chatgpt.com
- 37 LanceDB - LanceDB
https://docs.lancedb.com/?utm_source=chatgpt.com
- 38 NetworkX — NetworkX documentation
https://networkx.org/?utm_source=chatgpt.com
- 39 Neo4j Documentation
https://neo4j.com/docs/?utm_source=chatgpt.com
- 40 54 Project GraphRAG - Microsoft Research
https://www.microsoft.com/en-us/research/project/graphrag/?utm_source=chatgpt.com
- 44 Writing effective tools for AI agents—using AI agents \ Anthropic
<https://www.anthropic.com/engineering/writing-tools-for-agents>
- 45 Results - OpenAI Agents SDK
<https://openai.github.io/openai-agents-python/results/>
- 49 Demystifying evals for AI agents \ Anthropic
<https://www.anthropic.com/engineering/demystifying-evals-for-ai-agents>

⁵⁶ sverweij/dependency-cruiser: Validate and visualize ...
https://github.com/sverweij/dependency-cruiser?utm_source=chatgpt.com